

Plano Arquitetural - FIAP-8MLET-GRUPO-4

Visão geral

Objetivo do sistema: Coletar (scraping) dados de livros, persistir em banco relacional e expor os dados via API endpoints de consulta, incluindo estatísticas, além de expor endpoints com suporte a ML e um dashboard de monitoramento.

Pipeline de dados

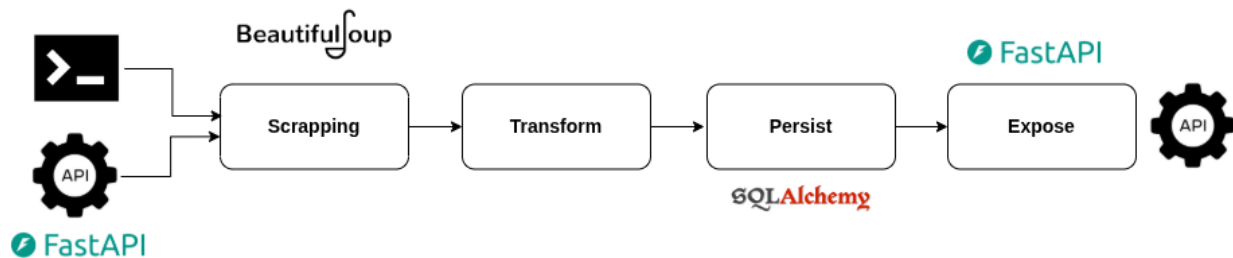


Figura 1. Pipeline de dados

O pipeline de dados pode ser compreendido em quatro etapas: ingestão, processamento, persistência e exposição. A ingestão é realizada através do scrapping utilizando a biblioteca BeautifulSoup, sendo que o pipeline de dados pode ser iniciado tanto via API, quanto via CLI. Os dados são processados em Python e persistidos no banco de dados PostgreSQL utilizando SQLAlchemy. Para consumo pelos usuários, os dados são disponibilizados a partir de endpoints utilizando FastAPI.

Arquitetura do sistema

O sistema apresentado possui arquitetura com separação de responsabilidades, permitindo maior manutenibilidade e escalabilidade para o sistema. A camada de API é composta pelos routers, classes responsáveis por definir os endpoints REST do sistema. Os dados recebidos pela API são disponibilizados a partir da Data transfer layer, que possui os schemas, ou seja, os objetos DTOs. A camada de serviço concentra toda a regra de negócio nas classes service, sendo essa a camada responsável pela persistência em banco de dados e a camada de dados é responsável pelo ORM e migrações do esquema de dados. Os componentes transversais como autenticação, logs e métricas são de responsabilidade da camada Cross-cutting (transversal), pois os mesmos são utilizados em diversas partes do sistema. Além disso, a camada CLI possui os scripts que são executados diretamente na linha

de comando e os testes automatizados executados no processo de CI/CD estão na camada CICD.

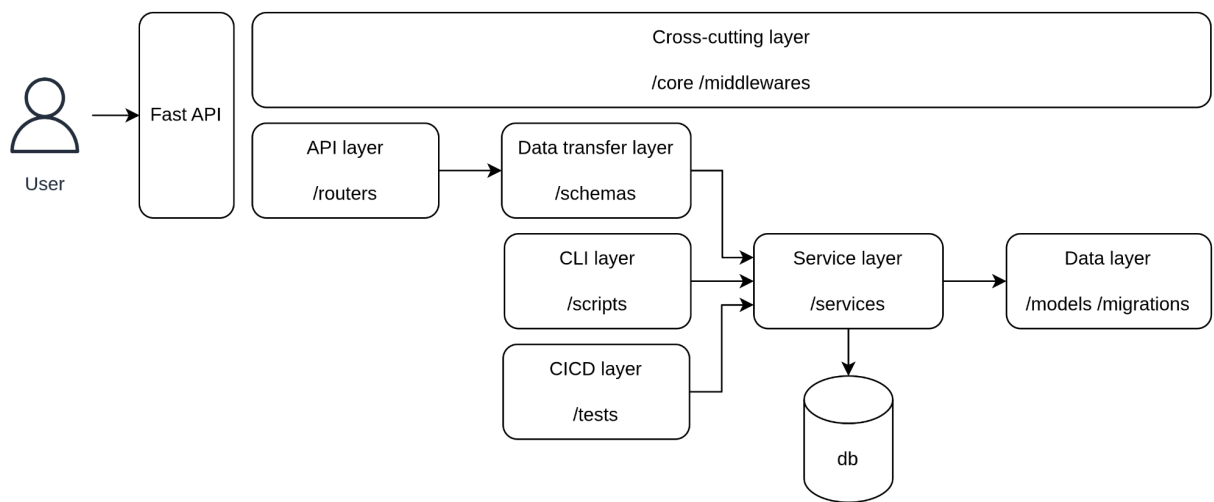


Figura 2. Arquitetura do sistema

A arquitetura apresentada permite escalabilidade futura tanto a nível técnico quanto funcional. A escalabilidade funcional pode se dar através da reutilização e inclusão de novas classes na service layer, sendo a separação da camada de serviço permite o acoplamento de novas camadas de visualização facilmente. A escalabilidade a nível técnico pode se dar pela execução de diferentes instâncias da API em paralelo, o que é permitido pela disponibilização via docker, sendo que se necessário é possível disponibilizar as camadas como serviços em um cluster kubernetes, por exemplo, de maneira a permitir execuções mais robustas e escaláveis. Para futura escalabilidade da persistência de dados, é possível isolar a camada de persistência, desacoplando a execução das consultas e persistência da camada service, permitindo a utilização de diversos bancos de dados. Além disso, a separação da camada de CI/CD facilita a adaptação futura para pipelines mais complexos.

Modelo Entidade-Relacionamento

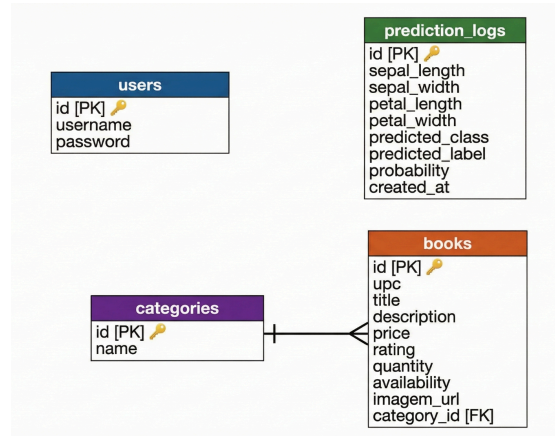


Figura 3. Modelo ER

O banco de dados utilizado no tech-challenge está em PostgreSQL 17 possui quatro entidades:

- Users: armazena informações de autenticação e identificação dos usuários do sistema.
- Books: armazena os dados principais dos livros coletados pelo processo de scrapping.
- Categories: armazena as categorias dos livros disponíveis na tabela books.
- Prediction_logs: armazena os dados associados às predições realizadas pelos modelos de machine learning do sistema

Cenários de uso para Cientistas de Dados

O sistema apresenta duas opções de utilização no contexto de ML, um modelo de classificação aplicado ao dataset Íris e um modelo de regressão aplicado ao domínio do negócio. O modelo Iris atua como um caso de referência para validação do pipeline de ML, enquanto o modelo de previsão de preços representa uma aplicação prática orientada ao domínio do negócio.

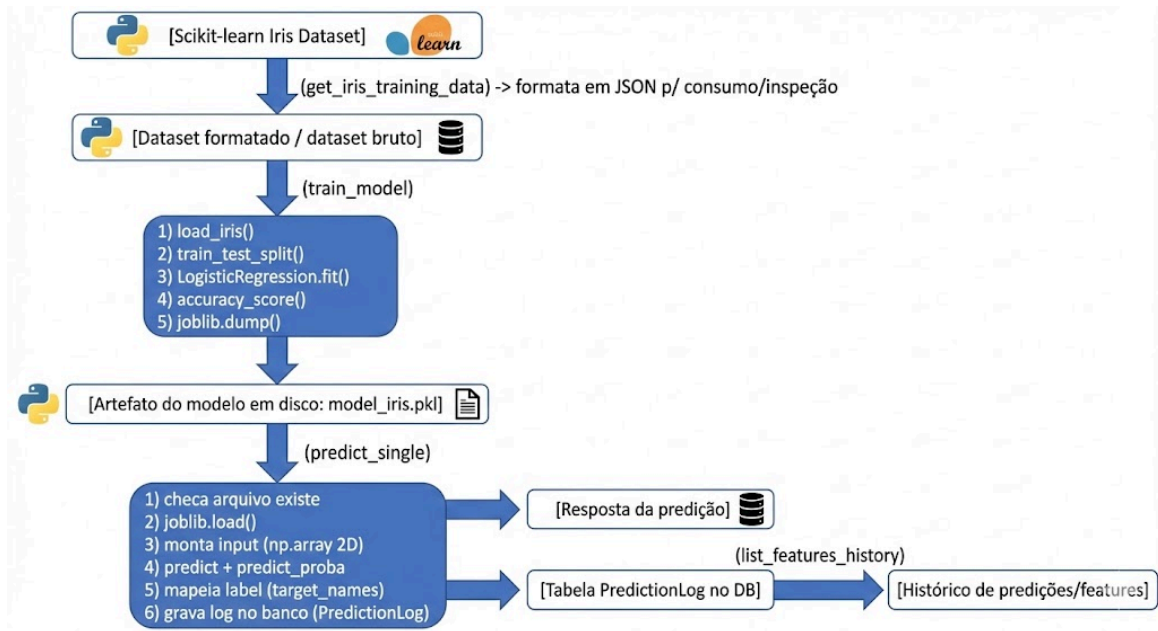


Figura 4. Visão geral do modelo de classificação

O modelo de classificação permite que o cientista de dados execute etapas de treinamento, avaliação e inferência utilizando o dataset público Iris, além de registrar todas as predições realizadas para fins de auditoria e análise de desempenho. São disponibilizados quatro endpoints com as seguintes funcionalidades:

- /api/v1/ml/training-data: retorna o dataset Iris completo formatado para treinamento.
- /api/v1/ml/train-model: dispara o retreinamento do modelo com os dados atuais e salva o arquivo .pkl no servidor.
- /api/v1/ml/predictions: recebe as medidas de uma flor Iris, retorna a espécie prevista pelo modelo e salva o log no banco de dados.
- /api/v1/ml/features: retorna o histórico de features (dados de entrada) submetidos à API de predição.

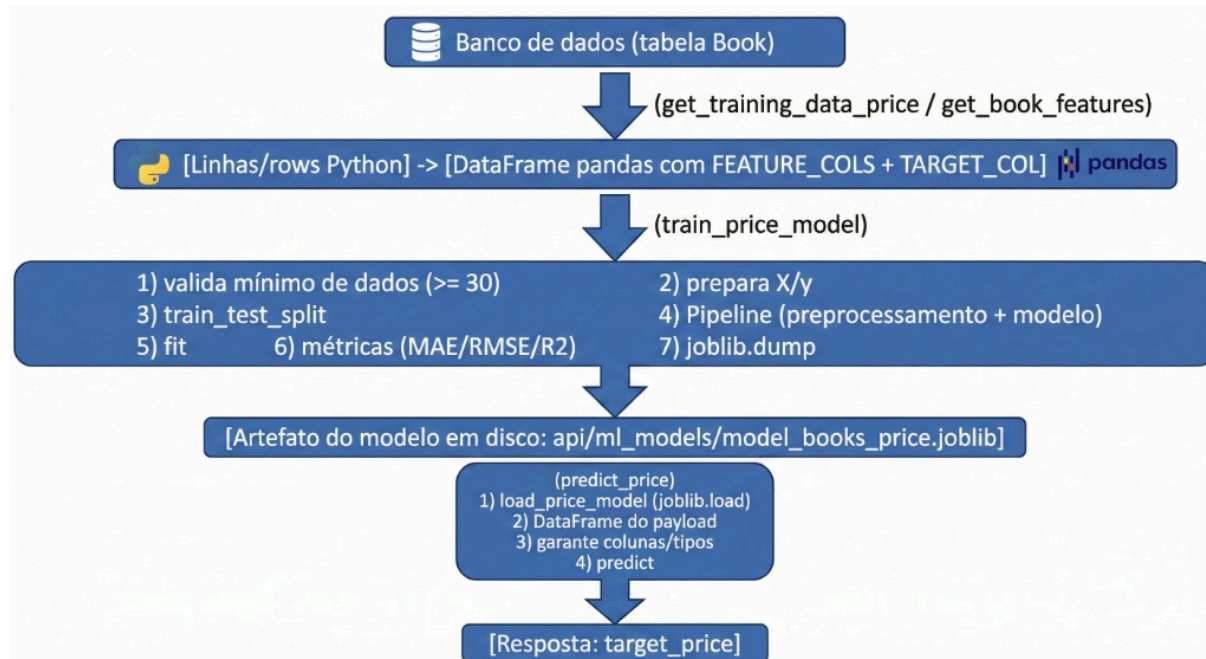


Figura 5. Visão geral do modelo de regressão

O modelo de regressão permite a previsão de preços de livros com base em dados coletados via scraping e armazenados no banco de dados. Nesse caso, o treinamento do modelo ocorre a partir das informações do PostgreSQL, permitindo experimentos baseados em dados atualizados e alinhados ao contexto da aplicação.

- `/api/v1/ml/books/features`: retorna as features dos livros já limpas e padronizadas, sem o preço, para inspeção e testes de inferência.
- `/api/v1/ml/books/training-data`: retorna o conjunto de dados rotulado (features + preço) utilizado como base para o treinamento do modelo.
- `/api/v1/ml/books/train-model`: executa o pipeline completo de treinamento do modelo de precificação e salva o modelo treinado em disco.
- `/api/v1/ml/books/prediction`: realiza a inferência do preço estimado de um livro a partir das features fornecidas, usando o modelo treinado

Como plano de integração de novos modelos de ML, novos modelos podem ser integrados na camada de serviços, onde são implementadas rotinas de treinamento, avaliação, persistência e inferência. Na arquitetura proposta, os modelos são tratados como artefatos independentes, armazenados em formato serializado (joblib), o que permite sua substituição, atualização ou expansão sem impacto direto na API ou na camada de dados. Essa abordagem possibilita a inclusão de novos modelos no futuro, inclusive há a possibilidade das funcionalidades de ML via linha de comando a partir da integração de novos scripts na camada CLI.